



CIS5560 Term Project Tutorial



Authors: Sirisha Mahesh

Instructor: [Jongwook Woo](#)

Date: 05/16/2025

Lab Tutorial

Sirisha Mahesh (smahesh4@calstatela.edu)

05/16/2025

Redfin Housing Project Analysis Predictive Analysis using Machine Learning Regression Models in Spark ML

Objectives

The Objective of the lab is to build a model that predicts the net price of housing market based on features like inventory, listings, homes sold etc.

Housing Market Prediction

- 1) Decision Tree Regression
- 2) Gradient Boost Tree Regression

Platform Specifications

- Hadoop Version – 3.3.3
- Pyspark Version – 3.2.1
- CPU Speed: 1995.309 MHz
- # of CPU cores : 8
- # of nodes: 5(2 master nodes, 3 worker nodes)
- Total Memory Size: 806.4 GB

Dataset Specifications

Dataset Name: <https://www.kaggle.com/datasets/thuynyle/redfin-housing-market-data>

Dataset Format: TVS [Converted to CSV using Python Code]

Dataset Size: 8.5GB +

Step 1: Get Data manually from the Data Source

We first need to download the dataset from Kaggle

<https://www.kaggle.com/datasets/thuynyle/redfin-housing-market-data>

Step 2 : Uploading data to Hadoop File System (HDFS)

Manually uploading the file to the Hadoop directory, we need to first transfer it to the local directory using scp command.

```
scp /d/MSIS-CSULA/CIS5560-Big-Data-Science/TermProject/CIS5560-  
Project/MyProejct/Redfine/Converted_To_CSV/merged_market_tracker.csv  
smahesh4@144.24.13.0:~
```

```
Sirisha@LAPTOP-8FCHOK16 MINGW64 ~ (main)  
$ scp /d/MSIS-CSULA/CIS5560-Big-Data-Science/TermProject/CIS5560-Project/MyProject/Redfine/Converted_To_CSV/merged_market_tracker.csv  
smahesh4@144.24.13.0:~  
smahesh4@144.24.13.0's password:  
merged_market_tracker.csv 100% 11GB 2.7MB/s 1:11:38  
  
Sirisha@LAPTOP-8FCHOK16 MINGW64 ~ (main)  
$ |
```

Step 3: Connect to Hadoop Spark Cluster

For that now open another shell terminal and paste the ssh command to connect to the Hadoop Spark Cluster

```
$ ssh smahesh4@144.24.13.0
```

Now enter the password same as username and connect to Hadoop cluster

Step 4: Create a directory “Project” in my case I have given the directory name as “Sirisha-Project” to put the file to HDFS

a) Run the following HDFS commands to create the directory in HDFS

```
Hdfs dfs -mkdir Sirisha-Project
```

b) Next, you can run the following shell command to put file in respective directory

```
Hdfs dfs -put merged_market_tracker.csv Sirisha-Project/
```

```
-bash-4.2$ hdfs dfs -mkdir Sirisha-Project
-bash-4.2$ hdfs dfs -put merged_market_tracker.csv Sirisha-Project/
-bash-4.2$ hdfs dfs -ls
Found 4 items
drwx----- - smahesh4 hdfs          0 2025-05-08 12:00 .Trash
drwxr-xr-x - smahesh4 hdfs          0 2025-05-05 14:38 .sparkStaging
drwxr-xr-x - smahesh4 hdfs          0 2025-05-16 01:20 Sirisha-Project
drwxr-xr-x - smahesh4 hdfs          0 2025-05-09 06:09 kaggle
-bash-4.2$ hdfs dfs -ls Sirisha-Project
Found 1 items
-rw-r--r--  3 smahesh4 hdfs 12295932640 2025-05-16 01:20 Sirisha-Project/merged_market_tracker.csv
-bash-4.2$ |
```

c) Run the following 2 HDFS commands to make sure if merged_market_tracker.csv is uploaded to the project directory

```
Hdfs dfs -ls
```

```
Sirisha@LAPTOP-8FCHOK16 MINGW64 ~ (main)
$ ssh smahesh4@144.24.13.0
smahesh4@144.24.13.0's password:
Last login: Fri May 16 00:19:58 2025 from syn-071-093-198-102.res.spectrum.com
-bash-4.2$ hdfs dfs -ls
Found 5 items
drwx----- - smahesh4 hdfs          0 2025-05-08 12:00 .Trash
drwxr-xr-x - smahesh4 hdfs          0 2025-05-16 02:55 .sparkStaging
-rw-r--r--  3 smahesh4 hdfs      3730 2025-05-16 02:27 Decision_Tree.py
drwxr-xr-x - smahesh4 hdfs          0 2025-05-16 01:20 Sirisha-Project
drwxr-xr-x - smahesh4 hdfs          0 2025-05-09 06:09 kaggle
-bash-4.2$ |
```

d) This command will display the list of all files in hdfs

```
Hdfs dfs -ls Sirisha-Project
```

```
-bash-4.2$ hdfs dfs -ls Sirisha-Project
Found 1 items
-rw-r--r--  3 smahesh4 hdfs 12295932640 2025-05-16 01:20 Sirisha-Project/merged_market_tracker.csv
-bash-4.2$
```

Step 5: Login to Databricks

Create the Cluster

The screenshot shows the Databricks 'Create New Cluster' interface. The left sidebar contains navigation links: 'New', 'Workspace', 'Recents', 'Search', 'Catalog', 'Workflows', 'Compute' (highlighted), 'Machine Learning', and 'Experiments'. The main content area is titled 'Compute' and shows the cluster name 'CIS5560-FinalProject'. Below the name are tabs for 'Configuration', 'Notebooks (0)', 'Libraries', 'Event log', 'Spark UI', 'Driver logs', 'Metrics', 'Apps', and 'Spark compute UI - Master'. The 'Configuration' tab is active, displaying settings for 'Databricks Runtime Version' (12.2 LTS), 'Driver type' (Community Optimized), and 'Instance' (Free 15 GB Memory). The 'Spark' section shows 'Spark config' and 'Environment variables'. Buttons for 'Terminate' and 'Edit' are in the top right.

Compute

CIS5560-FinalProject

Configuration Notebooks (0) Libraries Event log Spark UI Driver logs Metrics Apps Spark compute UI - Master

Databricks Runtime Version

12.2 LTS (includes Apache Spark 3.3.2, Scala 2.12)

Driver type

Community Optimized 15.3 GB Memory, 2 Cores

Instance

Free 15 GB Memory: As a Community Edition user, your compute will automatically terminate after an idle period of one or two hours. For more configuration options [click here](#), please upgrade your Databricks subscription. [click here](#)

Spark JDBC/ODBC

Spark config

spark.databricks.rocksDB.fileManager.useCommitService false

Environment variables

PYSPARK_PYTHON=/databricks/python3/bin/python3

Upload the File into the Databricks

The screenshot shows the Databricks 'Create New Table' interface. The left sidebar is identical to the previous screenshot, with 'Compute' highlighted. The main content area is titled 'Add data' and 'Create New Table'. Below the title are tabs for 'Data source', 'Upload File' (highlighted), and 'S3'. The 'Upload File' tab is active, displaying a 'DBFS Target Directory' field with the value '/FileStore/tables/' and a 'Select' button. Below this is a message: 'Files uploaded to DBFS are accessible by everyone who has access to this workspace. [Learn more](#)'. The 'Files' section shows a file named 'market_tracker.csv' being uploaded, with a progress bar and a 'Cancel upload' button.

data

Create New Table

Data source

Upload File S3

DBFS Target Directory

/FileStore/tables/ (optional) Select

Files uploaded to DBFS are accessible by everyone who has access to this workspace. [Learn more](#)

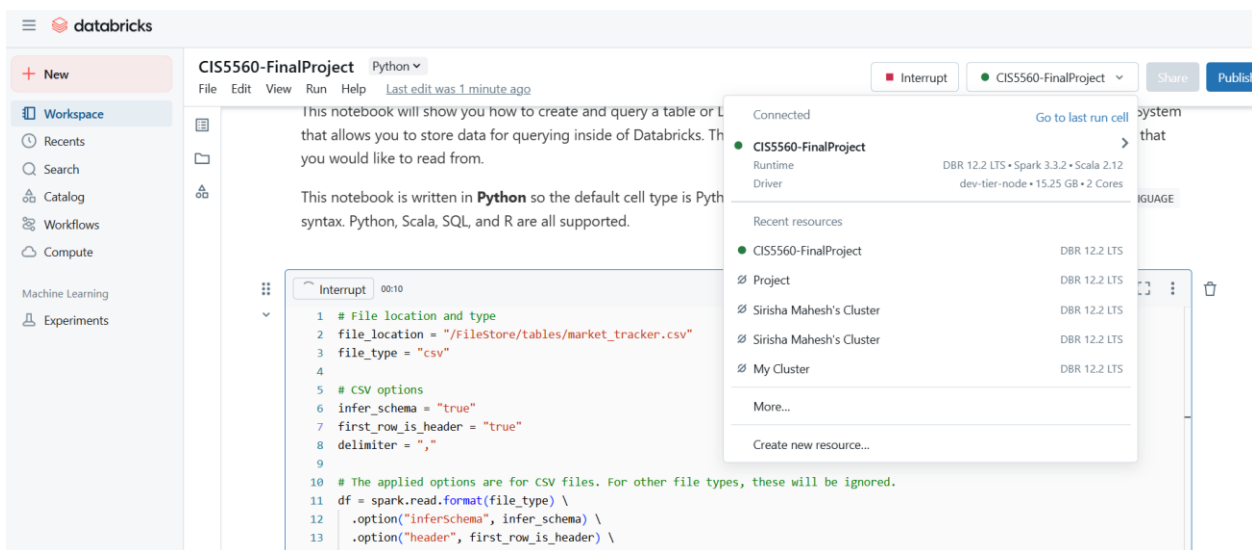
Files

market_tracker.csv

0.3 GB

Cancel upload

Once the Data is loaded connect to the created Cluster



Step 6 : Data Loading and Cleaning

This code cleans string-formatted numeric columns in a PySpark DataFrame by removing unwanted characters (commas and dollar signs), converting them to numeric (double) type, and dropping any resulting null values.

Code

```
from pyspark.sql.functions import regexp_replace
numeric_cols = [
    "HOMES_SOLD", "NEW_LISTINGS", "INVENTORY", "MONTHS_OF_SUPPLY",
    "MEDIAN_LIST_PRICE", "MEDIAN_PPSF", "PENDING_SALES", "MEDIAN_SALE_PRICE"
]
for c in numeric_cols:
    df = df.withColumn(c, regexp_replace(col(c), ",", ""))
    df = df.withColumn(c, regexp_replace(col(c), "\\$", ""))
    df = df.withColumn(c, col(c).cast("double"))
df = df.dropna(subset=numeric_cols)
```

```
09:52 PM (1s) 3

# Data Cleaning

from pyspark.sql.functions import regexp_replace

numeric_cols = [
    "HOMES_SOLD", "NEW_LISTINGS", "INVENTORY", "MONTHS_OF_SUPPLY",
    "MEDIAN_LIST_PRICE", "MEDIAN_PPSF", "PENDING_SALES", "MEDIAN_SALE_PRICE"
]

for c in numeric_cols:
    df = df.withColumn(c, regexp_replace(col(c), ",", ""))
    df = df.withColumn(c, regexp_replace(col(c), "\\$", ""))
    df = df.withColumn(c, col(c).cast("double"))

df = df.dropna(subset=numeric_cols)

df: pyspark.sql.dataframe.DataFrame = [PERIOD_BEGIN: date, PERIOD_END: date ... 57 more fields]
```

Step 7: Feature Engineering

This code prepares the dataset for machine learning by assembling selected feature columns and applying normalization. It first defines a list of numeric columns (feature_cols) to be used as input features. Then, it uses VectorAssembler to combine these individual columns into a single feature vector named "features". Next, it applies MinMaxScaler to scale the values in the "features" column to a normalized range (typically 0 to 1), storing the result in a new column "normFeatures". Finally, it creates a cleaned DataFrame called final_data that includes all feature columns and the target variable "MEDIAN_SALE_PRICE" (renamed as "label"), dropping any rows with null values. This structured and scaled data is ready for training machine learning models.

Code

```
from pyspark.ml.feature import VectorAssembler, MinMaxScaler
feature_cols = [
    "HOMES_SOLD", "NEW_LISTINGS", "INVENTORY",
    "MONTHS_OF_SUPPLY", "MEDIAN_LIST_PRICE",
    "MEDIAN_PPSF", "PENDING_SALES"
]
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
scaler = MinMaxScaler(inputCol="features", outputCol="normFeatures")
final_data = df.select(*feature_cols, col("MEDIAN_SALE_PRICE").alias("label")).dropna()
```

```
09:53 PM (<1s) 4

### Feature Engineering

#We use VectorAssembler to combine input columns into a single feature vector. We then scale the features using MinMaxScaler.
from pyspark.ml.feature import VectorAssembler, MinMaxScaler

feature_cols = [
    "HOMES_SOLD", "NEW_LISTINGS", "INVENTORY",
    "MONTHS_OF_SUPPLY", "MEDIAN_LIST_PRICE",
    "MEDIAN_PPSF", "PENDING_SALES"
]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
scaler = MinMaxScaler(inputCol="features", outputCol="normFeatures")

final_data = df.select(*feature_cols, col("MEDIAN_SALE_PRICE").alias("label")).dropna()
```

final_data: pyspark.sql.dataframe.DataFrame = [HOMES_SOLD: double, NEW_LISTINGS: double ... 6 more fields]

Step 8 : Split Data

This code splits the prepared dataset into training and testing sets for machine learning. Using `randomSplit`, it divides `final_data` into two parts: 80% for training (`train_data`) and 20% for testing (`test_data`). The `seed=42` ensures the split is reproducible, meaning the same split will occur each time the code is run with this seed. This setup allows for training a model on one portion of the data and evaluating its performance on unseen data.

Code

```
#Split Data
splits = final_data.randomSplit([0.8, 0.2], seed=42)
train_data = splits[0]
test_data = splits[1]
```

```
09:54 PM (<1s) 5

#Split Data
splits = final_data.randomSplit([0.8, 0.2], seed=42)
train_data = splits[0]
test_data = splits[1]
```

test_data: pyspark.sql.dataframe.DataFrame = [HOMES_SOLD: double, NEW_LISTINGS: double ... 6 more fields]

train_data: pyspark.sql.dataframe.DataFrame = [HOMES_SOLD: double, NEW_LISTINGS: double ... 6 more fields]

Step 9 : Training NLP

This code prepares a text column for Natural Language Processing (NLP) by combining multiple categorical columns into a single string column named "DESCRIPTION". Using `concat_ws`, it joins the values from "REGION", "CITY", "STATE", and "PROPERTY_TYPE" into one space-separated string per row. This combined column captures descriptive location and property information, which can then be tokenized and cleaned (e.g., stop word removal) for NLP tasks like classification or clustering.

Code

NLP Pipeline (Tokenization and Stop Word Removal)

#This step is to tokenize text data and remove stop words using PySpark. We use the ``RegexTokenizer`` to split the text into alphabetic tokens of minimum 3 characters. Then we remove common stop words using ``StopWordsRemover``. This prepares the text column for NLP analysis.

```
from pyspark.sql.functions import concat_ws
```

```
# Combine REGION, CITY, STATE, and PROPERTY_TYPE into a single text column
df = df.withColumn("DESCRIPTION", concat_ws(" ", "REGION", "CITY", "STATE",
"PROPERTY_TYPE"))
```

```
09:55 PM (<1s) 6

### NLP Pipeline (Tokenization and Stop Word Removal)

#This step is to tokenize text data and remove stop words using PySpark. We use the `RegexTokenizer` to split the text into
alphabetic tokens of minimum 3 characters. Then we remove common stop words using `StopWordsRemover`. This prepares the text
column for NLP analysis.

from pyspark.sql.functions import concat_ws

# Combine REGION, CITY, STATE, and PROPERTY_TYPE into a single text column
df = df.withColumn("DESCRIPTION", concat_ws(" ", "REGION", "CITY", "STATE", "PROPERTY_TYPE"))

df: pyspark.sql.dataframe.DataFrame = [PERIOD_BEGIN: date, PERIOD_END: date ... 57 more fields]
```

Step 10: NLP Preprocessing: Tokenization Step

This code uses PySpark's `RegexTokenizer` to convert the combined text column "DESCRIPTION" into an array of meaningful word tokens for NLP tasks. It extracts alphabetic word sequences (using the regex `\p{L}+`) that are at least 3 characters long, filtering out short or less informative words. The tokenizer processes the "DESCRIPTION" column and outputs the results into a new column called "words". Finally, the code displays the original descriptions alongside their corresponding tokenized words for the first five rows, allowing a quick check of the tokenization output.

Code

```
# Tokenize Text Using RegexTokenizer
from pyspark.ml.feature import RegexTokenizer

# Tokenize using regex: alphabetic sequences, min length 3
tokenizer = RegexTokenizer() \
    .setPattern("\\p{L}+") \
    .setMinTokenLength(3) \
    .setGaps(False) \
    .setInputCol("DESCRIPTION") \
    .setOutputCol("words")

tokenized_df = tokenizer.transform(df)

# Show sample tokens
tokenized_df.select("DESCRIPTION", "words").show(5, truncate=False)
```

```
09:56 PM (2s) 7

# Tokenize Text Using RegexTokenizer
from pyspark.ml.feature import RegexTokenizer

# Tokenize using regex: alphabetic sequences, min length 3
tokenizer = RegexTokenizer() \
    .setPattern("\\p{L}+") \
    .setMinTokenLength(3) \
    .setGaps(False) \
    .setInputCol("DESCRIPTION") \
    .setOutputCol("words")

tokenized_df = tokenizer.transform(df)

# Show sample tokens
tokenized_df.select("DESCRIPTION", "words").show(5, truncate=False)
```

▶ (1) Spark Jobs

tokenized_df: pyspark.sql.dataframe.DataFrame = [PERIOD_BEGIN: date, PERIOD_END: date ... 58 more fields]

tokenized_df: pyspark.sql.dataframe.DataFrame = [PERIOD_BEGIN: date, PERIOD_END: date ... 58 more fields]

DESCRIPTION	words
Lewisburg, PA metro area All Residential	[lewisburg, metro, area, all, residential]
Philadelphia, PA metro area All Residential	[philadelphia, metro, area, all, residential]
California, MD metro area Single Family Residential	[california, metro, area, single, family, residential]
Corvallis, OR metro area Townhouse	[corvallis, metro, area, townhouse]
Hartford, CT metro area Multi-Family (2-4 Unit)	[hartford, metro, area, multi, family, unit]

only showing top 5 rows

Step 11: NLP Preprocessing: Stop Words Removal

This code removes common English stop words (like "the", "is", "in", etc.) from the tokenized text to retain only the most meaningful words for NLP tasks. It uses PySpark's `StopWordsRemover`, specifying the input column "words" (which contains tokenized text) and the output column "filtered" (which will store the cleaned tokens). The transformation is applied to `tokenized_df`, resulting in `filtered_df`, which contains both the original and cleaned tokens. Finally, the code displays the first five rows to compare the original and filtered word lists.

Code

```
# Remove Stop Words
from pyspark.ml.feature import StopWordsRemover

# Use built-in English stop words
remover = StopWordsRemover(inputCol="words", outputCol="filtered")
filtered_df = remover.transform(tokenized_df)

# Show cleaned tokens
filtered_df.select("words", "filtered").show(5, truncate=False)
```



The screenshot shows a PySpark notebook interface. At the top, there's a status bar with a green checkmark, the time '09:56 PM (1s)', and a page number '8'. Below this, the code from the previous block is displayed with syntax highlighting. The code defines a `StopWordsRemover` object and applies it to `tokenized_df` to create `filtered_df`. Below the code, a section titled '(1) Spark Jobs' shows the execution of the `transform` operation. At the bottom, a preview of the `filtered_df` DataFrame is shown. It has two columns: `words` and `filtered`. The preview displays five rows of data, showing the removal of stop words like 'all', 'single', 'multi', and 'unit' from the original token lists.

```
# Remove Stop Words
from pyspark.ml.feature import StopWordsRemover

# Use built-in English stop words
remover = StopWordsRemover(inputCol="words", outputCol="filtered")
filtered_df = remover.transform(tokenized_df)

# Show cleaned tokens
filtered_df.select("words", "filtered").show(5, truncate=False)
```

► (1) Spark Jobs

► filtered_df: pyspark.sql.dataframe.DataFrame = [PERIOD_BEGIN: date, PERIOD_END: date ... 59 more fields]

words	filtered
[[lewisburg, metro, area, all, residential]]	[[lewisburg, metro, area, residential]]
[[philadelphia, metro, area, all, residential]]	[[philadelphia, metro, area, residential]]
[[california, metro, area, single, family, residential]]	[[california, metro, area, single, family, residential]]
[[corvallis, metro, area, townhouse]]	[[corvallis, metro, area, townhouse]]
[[hartford, metro, area, multi, family, unit]]	[[hartford, metro, area, multi, family, unit]]

Step 12: Topic Modeling Using LDA

This step applies topic modeling to the cleaned text data using PySpark's LDA (Latent Dirichlet Allocation) algorithm. First, it converts the filtered tokens into numerical feature vectors using `CountVectorizer`, which transforms each document into a sparse vector of token counts. The resulting `vectorized_df` is then used to train an LDA model with `k=5` topics and up to 10 iterations. After training, `lda_model.describeTopics()` extracts the top words (terms) associated with each topic. The vocabulary is retrieved from the vectorizer to map

the term indices back to actual words. Finally, the code displays the topics and prints out the top keywords representing each topic, allowing interpretation of the themes present in the text data.

Code

```
# Topic Modeling with LDA
from pyspark.ml.feature import CountVectorizer
from pyspark.ml.clustering import LDA

# Convert words to vectors
vectorizer = CountVectorizer(inputCol="filtered", outputCol="features")
vectorized_df = vectorizer.fit(filtered_df).transform(filtered_df)

# Fit LDA model
lda = LDA(k=5, maxIter=10)
lda_model = lda.fit(vectorized_df)

# Describe topics
topics = lda_model.describeTopics()
vocab = vectorizer.fit(filtered_df).vocabulary

topics.show()

# Print top words per topic
for row in topics.collect():
    print(f"Topic {row['topic']}: {[vocab[i] for i in row['termIndices']]}")
```

```
# Topic Modeling with LDA
from pyspark.ml.feature import CountVectorizer
from pyspark.ml.clustering import LDA

# Convert words to vectors
vectorizer = CountVectorizer(inputCol="filtered", outputCol="features")
vectorized_df = vectorizer.fit(filtered_df).transform(filtered_df)

# Fit LDA model
lda = LDA(k=5, maxIter=10)
lda_model = lda.fit(vectorized_df)

# Describe topics
topics = lda_model.describeTopics()
vocab = vectorizer.fit(filtered_df).vocabulary

topics.show()

# Print top words per topic
for row in topics.collect():
    print(f"Topic {row['topic']}: {[vocab[i] for i in row['termIndices']]}")
```

▶ (26) Spark Jobs

▶ topics: pyspark.sql.dataframe.DataFrame = [topic: integer, termIndices: array ... 1 more field]
 ▶ vectorized_df: pyspark.sql.dataframe.DataFrame

```
+-----+-----+-----+
|topic|      termIndices|      termWeights|
+-----+-----+-----+
|  0|[1, 0, 2, 3, 4, 1...|[0.18248339164969...|
|  1|[0, 1, 2, 3, 4, 8...|[0.20667201326196...|
|  2|[0, 1, 5, 9, 2, 1...|[0.22600192605532...|
|  3|[0, 1, 2, 3, 4, 9...|[0.19209640314789...|
|  4|[1, 0, 3, 7, 6, 8...|[0.18669142435319...|
+-----+-----+-----+
```

```
Topic 0: ['area', 'metro', 'residential', 'family', 'single', 'falls', 'salem', 'las', 'college', 'washington']
Topic 1: ['metro', 'area', 'residential', 'family', 'single', 'townhouse', 'new', 'fort', 'mount', 'rapids']
Topic 2: ['metro', 'area', 'condo', 'city', 'residential', 'springs', 'townhouse', 'jamestown', 'bay', 'green']
Topic 3: ['metro', 'area', 'residential', 'family', 'single', 'city', 'lake', 'gainesville', 'jacksonville', 'condo']
Topic 4: ['area', 'metro', 'family', 'unit', 'multi', 'townhouse', 'condo', 'residential', 'city', 'san']
```

Step 13: SQL Queries

```
02:00 PM (<1s) 10
df.createOrReplaceTempView("market_data")
```

```
df.createOrReplaceTempView("market_data")
```

a) Regional Price Statistics: This SQL query analyzes housing market data by region, summarizing key statistics for the MEDIAN_SALE_PRICE column. It runs on a temporary table called market_data, which was likely created from a Spark DataFrame. The query groups records by region and computes COUNT(*): Number of records per region, AVG(MEDIAN_SALE_PRICE): Average sale price, MAX(MEDIAN_SALE_PRICE): Highest sale price, MIN(MEDIAN_SALE_PRICE): Lowest sale price, STDDEV(MEDIAN_SALE_PRICE): Standard deviation to measure price variability. It filters out rows with null sale prices, sorts the results by average price in descending order, and provides a ranked view of the most expensive regions based on median sale prices.

Code

```
%sql

/* Query the created temp table in a SQL cell */

SELECT
  region,
  COUNT(*) AS records,
  AVG(MEDIAN_SALE_PRICE) AS avg_price,
  MAX(MEDIAN_SALE_PRICE) AS max_price,
  MIN(MEDIAN_SALE_PRICE) AS min_price,
  STDDEV(MEDIAN_SALE_PRICE) AS price_stddev
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL
GROUP BY region
ORDER BY avg_price DESC
```

```

09:59 PM (10s) 10

%sql

/* Query the created temp table in a SQL cell */

SELECT
  region,
  COUNT(*) AS records,
  AVG(MEDIAN_SALE_PRICE) AS avg_price,
  MAX(MEDIAN_SALE_PRICE) AS max_price,
  MIN(MEDIAN_SALE_PRICE) AS min_price,
  STDDEV(MEDIAN_SALE_PRICE) AS price_stddev
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL
GROUP BY region
ORDER BY avg_price DESC

```

▶ (2) Spark Jobs

▶ _sqldf: pyspark.sql.dataframe.DataFrame = [region: string, records: long ... 4 more fields]

▶ _sqldf: pyspark.sql.dataframe.DataFrame = [region: string, records: long ... 4 more fields]

	region	records	avg_price	max_price	min_price	price_stddev
1	San Francisco, CA metro area	795	1269740.59182389...	996500	1000000	364866.49383428396
2	San Rafael, CA metro area	703	1098479.67567567...	997500	1000000	437102.1816065571
3	Vineyard Haven, MA metro area	449	1027752.35412026...	999518.5	1000000	668831.9092629342
4	San Jose, CA metro area	795	1022815.33522012...	999500	1000000	385735.8867069645
5	Jackson, WY metro area	665	865424.9045112781	999999	1016250	567184.9107116665
6	Edwards, CO metro area	795	861845.6647798743	999000	1000000	499458.9896403354
7	Santa Cruz, CA metro area	795	861335.7550101799	997000	1000000	317403.8014046174

b) SQL Time-Series Analysis: Average Sale Price by Year and Region

This SQL query performs a time-series analysis of housing market trends by calculating the average MEDIAN_SALE_PRICE per year and region. It uses the %sql magic command (for Databricks or similar environments) to run SQL on the market_data temporary table.

YEAR(PERIOD_BEGIN): Extracts the year from the date column PERIOD_BEGIN.

GROUP BY year, region: Groups the data by year and region.

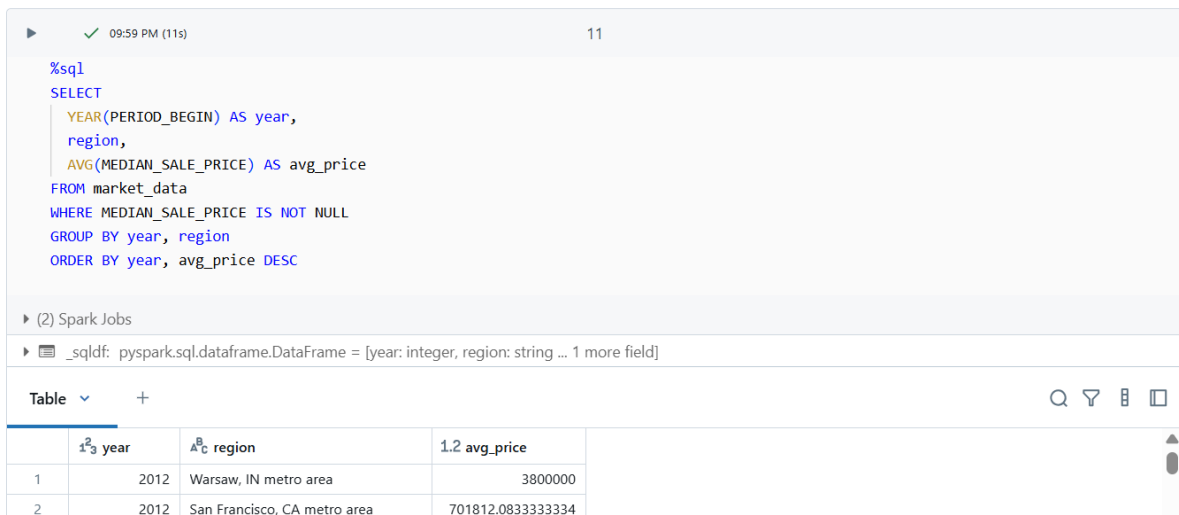
AVG(MEDIAN_SALE_PRICE): Calculates the average sale price for each year-region pair.

WHERE MEDIAN_SALE_PRICE IS NOT NULL: Excludes records with missing sale prices.

ORDER BY year, avg_price DESC: Sorts the results by year and within each year, ranks regions by descending average price.

This query is useful for identifying pricing trends over time and spotting the most expensive regions in each year.


```
%sql
SELECT
  YEAR(PERIOD_BEGIN) AS year,
  region,
  AVG(MEDIAN_SALE_PRICE) AS avg_price
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL
GROUP BY year, region
ORDER BY year, avg_price DESC
```



09:59 PM (11s) 11

```
%sql
SELECT
  YEAR(PERIOD_BEGIN) AS year,
  region,
  AVG(MEDIAN_SALE_PRICE) AS avg_price
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL
GROUP BY year, region
ORDER BY year, avg_price DESC
```

▸ (2) Spark Jobs

▸ _sqldf: pyspark.sql.dataframe.DataFrame = [year: integer, region: string ... 1 more field]

Table ▾ +

	1.2 year	1.1 region	1.2 avg_price
1	2012	Warsaw, IN metro area	3800000
2	2012	San Francisco, CA metro area	701812.0833333334

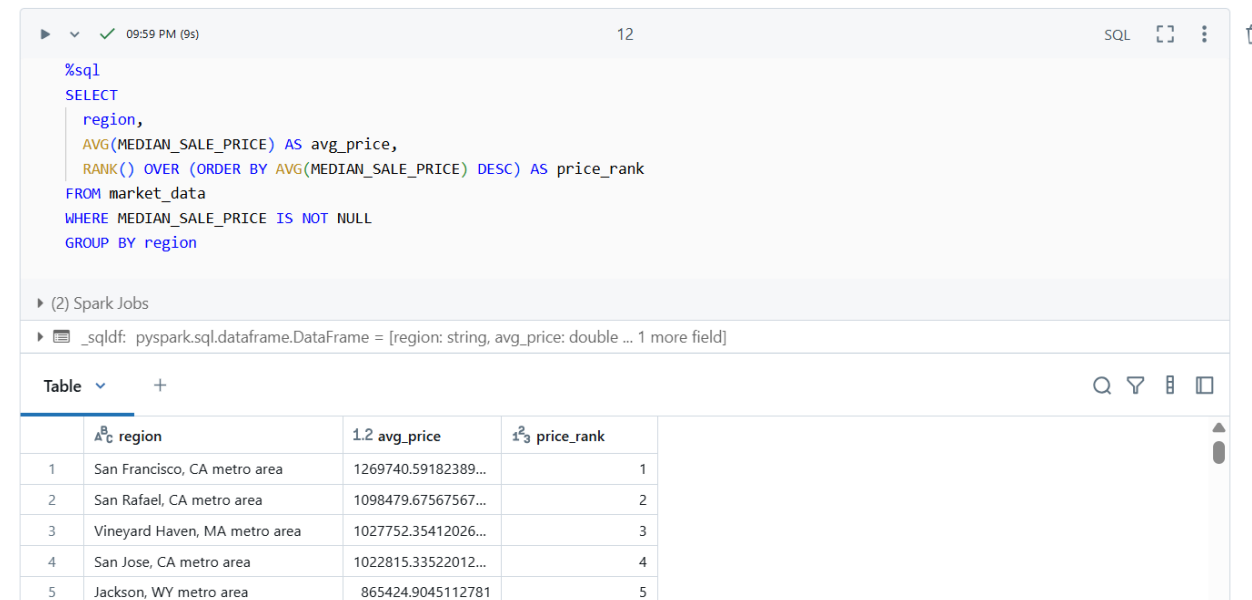
c) SQL Window Function: Ranking Regions by Average Sale Price

This SQL query ranks regions based on their average MEDIAN_SALE_PRICE using a window function. It operates on the market_data temporary table and performs the following:

- AVG(MEDIAN_SALE_PRICE):** Calculates the average sale price for each region.
- RANK() OVER (ORDER BY AVG(MEDIAN_SALE_PRICE) DESC):** Assigns a rank to each region based on its average price, with the highest average price ranked first. Regions with the same average price get the same rank (with gaps).
- BY region:** Groups the dataset by region to calculate the average.
- WHERE MEDIAN_SALE_PRICE IS NOT NULL:** Ensures only valid (non-null) prices are considered.

This query is useful for generating a leaderboard-style ranking of regions by housing market value.

```
%sql
SELECT
  region,
  AVG(MEDIAN_SALE_PRICE) AS avg_price,
  RANK() OVER (ORDER BY AVG(MEDIAN_SALE_PRICE) DESC) AS price_rank
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL
GROUP BY region
```



```
%sql
SELECT
  region,
  AVG(MEDIAN_SALE_PRICE) AS avg_price,
  RANK() OVER (ORDER BY AVG(MEDIAN_SALE_PRICE) DESC) AS price_rank
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL
GROUP BY region
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [region: string, avg_price: double ... 1 more field]

	1 region	1.2 avg_price	1.2 price_rank
1	San Francisco, CA metro area	1269740.59182389...	1
2	San Rafael, CA metro area	1098479.67567567...	2
3	Vineyard Haven, MA metro area	1027752.35412026...	3
4	San Jose, CA metro area	1022815.33522012...	4
5	Jackson, WY metro area	865424.9045112781	5

d) SQL Analysis: Comparing Average Sale Price and Inventory by Region

This SQL query analyzes the relationship between housing prices and inventory levels across different regions using the `market_data` temporary table. `AVG(MEDIAN_SALE_PRICE)`: Calculates the average sale price per region. `AVG(INVENTORY)`: Computes the average housing inventory per region. `WHERE MEDIAN_SALE_PRICE IS NOT NULL AND INVENTORY IS NOT NULL`: Filters out rows with missing price or inventory data to ensure accurate averages. `GROUP BY region`: Groups the data by region for aggregation. `ORDER BY avg_price DESC`: Sorts the results to show regions with the highest average sale prices at the top. This query helps identify whether higher-priced regions tend to have more or less inventory, which can offer insights into supply and demand trends in the housing market.

```
%sql
SELECT
  region,
  AVG(MEDIAN_SALE_PRICE) AS avg_price,
  AVG(INVENTORY) AS avg_inventory
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL AND INVENTORY IS NOT NULL
GROUP BY region
ORDER BY avg_price DESC
```

09:59 PM (9s) 13

```
%sql
SELECT
  region,
  AVG(MEDIAN_SALE_PRICE) AS avg_price,
  AVG(INVENTORY) AS avg_inventory
FROM market_data
WHERE MEDIAN_SALE_PRICE IS NOT NULL AND INVENTORY IS NOT NULL
GROUP BY region
ORDER BY avg_price DESC
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [region: string, avg_price: double ... 1 more field]

	region	1.2 avg_price	1.2 avg_inventory
1	San Francisco, CA metro area	1269740.59182389...	618.2716981132075
2	San Rafael, CA metro area	1098479.67567567...	216.01734104046244
3	Vineyard Haven, MA metro area	1027752.35412026...	76.72108843537416
4	San Jose, CA metro area	1022815.33522012...	669.7006289308176
5	Jackson, WY metro area	865424.9045112781	134.48493975903614

10:00 PM (9s) 15

```
%sql
SELECT * FROM clean_region_prices ORDER BY avg_price DESC;
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [region: string, avg_price: double]

	region	1.2 avg_price
1	San Francisco, CA metro area	1269740.59182389...
2	San Rafael, CA metro area	1098479.67567567...
3	Vineyard Haven, MA metro area	1027752.35412026...
4	San Jose, CA metro area	1022815.33522012...
5	Jackson, WY metro area	865424.9045112781
6	Edwards, CO metro area	861845.6647798743
7	Santa Cruz, CA metro area	803235.7560202788
8	Breckenridge, CO metro area	785315.1620185924
9	Heber, UT metro area	776147.0967016491

```
%sql
SELECT * FROM clean_region_prices ORDER BY avg_price DESC;
```

e) Visualizations

```
▶ 5 minutes ago (15s) 17

pandas_df = df.select("INVENTORY", "MEDIAN_SALE_PRICE") \
    .dropna() \
    .sample(False, 0.1) \
    .toPandas()

▶ (1) Spark Jobs
```

```
pandas_df = df.select("INVENTORY", "MEDIAN_SALE_PRICE") \
    .dropna() \
    .sample(False, 0.1) \
    .toPandas()
```

This code uses Seaborn and Matplotlib to visualize the relationship between housing inventory and median sale price. It creates a scatter plot with a regression trend line using `sns.regplot`, where `INVENTORY` is plotted on the x-axis and `MEDIAN_SALE_PRICE` on the y-axis. The `scatter_kws` parameter sets the point transparency to 30% for better visibility of overlapping points, and `line_kws` changes the regression line color to red. Finally, a title is added, the grid is enabled for readability, and the plot is displayed with `plt.show()`. This visualization helps to identify the trend or correlation between inventory and sale prices.

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.regplot(
    data=pandas_df,
    x="INVENTORY",
    y="MEDIAN_SALE_PRICE",
    scatter_kws={"alpha": 0.3},
    line_kws={"color": "red"} # Change line color here
)
plt.title("Trend: Inventory vs Median Sale Price")
```

```
plt.grid(True)
plt.show()
```

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.regplot(
    data=pandas_df,
    x="INVENTORY",
    y="MEDIAN_SALE_PRICE",
    scatter_kws={"alpha": 0.3},
    line_kws={"color": "red"} # Change line color here
)

plt.title("Trend: Inventory vs Median Sale Price")
plt.grid(True)
plt.show()
```

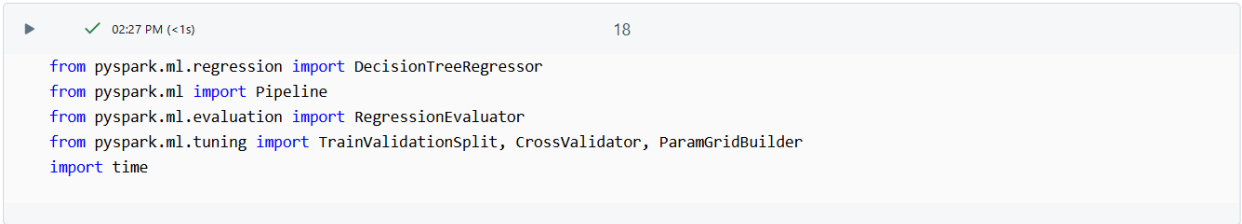


Step 14: Decision Tree Regressor

[i] Model Setup: Decision Tree with Tuning and Evaluation

This code sets up the necessary components to build, tune, and evaluate a Decision Tree Regression model in PySpark. It imports `DecisionTreeRegressor` to create the regression model and `Pipeline` to structure the machine learning workflow. `RegressionEvaluator` is used to measure model performance using metrics like RMSE or R^2 . For hyperparameter tuning, it imports `TrainValidationSplit` and `CrossValidator`, along with `ParamGridBuilder` to define a range of parameters to test. The `time` module is also imported, which can be used to measure the duration of training or evaluation processes.

```
from pyspark.ml.regression import DecisionTreeRegressor
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.tuning import TrainValidationSplit, CrossValidator,
ParamGridBuilder
import time
```



```
from pyspark.ml.regression import DecisionTreeRegressor
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.tuning import TrainValidationSplit, CrossValidator, ParamGridBuilder
import time
```

[ii] Pipeline Configuration with Decision Tree Regressor

This code defines a machine learning pipeline using a Decision Tree Regressor in PySpark. The `DecisionTreeRegressor` is initialized with `labelCol="label"` and `featuresCol="normFeatures"`, specifying the target and input feature columns. A `Pipeline` is then constructed, consisting of three stages: `assembler` (which combines features), `scaler` (which normalizes them), and `dt` (the decision tree model). This modular setup ensures that data preprocessing and model training are executed sequentially and consistently during both training and inference.

```
dt = DecisionTreeRegressor(labelCol="label", featuresCol="normFeatures")
pipeline = Pipeline(stages=[assembler, scaler, dt])
```

```
dt = DecisionTreeRegressor(labelCol="label", featuresCol="normFeatures")
pipeline = Pipeline(stages=[assembler, scaler, dt])
```

[iii] Hyperparameter Grid for Decision Tree

This code defines a hyperparameter grid for tuning the Decision Tree Regressor using PySpark's ParamGridBuilder. It specifies two hyperparameters to test: `maxDepth` (the maximum depth of the tree) with values 5 and 10, and `maxBins` (the number of bins used when discretizing continuous features) with values 32 and 64. The `.build()` method finalizes the grid into a list of parameter combinations. This grid will be used in model tuning methods like `TrainValidationSplit` or `CrossValidator` to find the best-performing configuration.

```
paramGrid = ParamGridBuilder() \
    .addGrid(dt.maxDepth, [5, 10]) \
    .addGrid(dt.maxBins, [32, 64]) \
    .build()
```

```
paramGrid = ParamGridBuilder() \
    .addGrid(dt.maxDepth, [5, 10]) \
    .addGrid(dt.maxBins, [32, 64]) \
    .build()
```

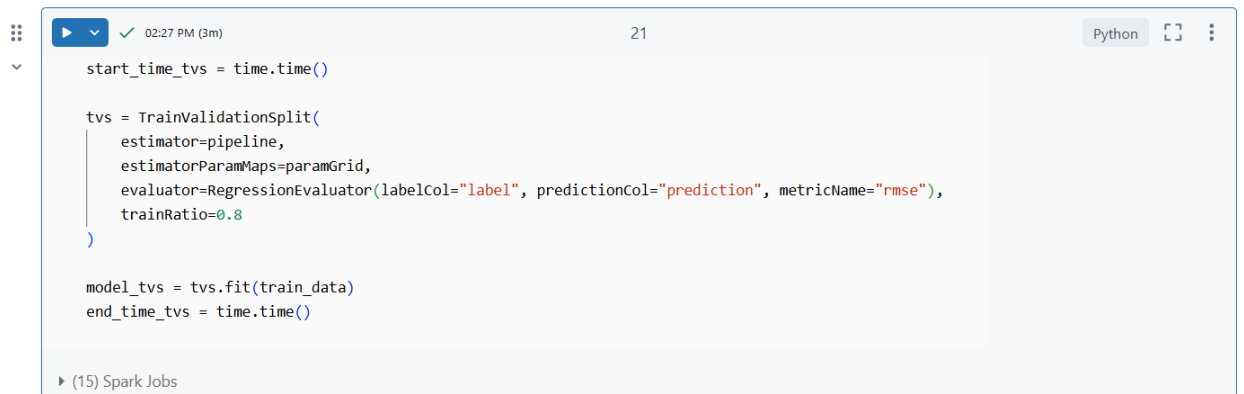
[iv] TrainValidationSplit for Model Tuning and Training Time Measurement

This code performs hyperparameter tuning and model training using PySpark's TrainValidationSplit while also measuring the training duration. It starts by capturing the start time with `time.time()`. Then, it sets up TrainValidationSplit with:

- `estimator=pipeline`: the full pipeline including feature engineering and the Decision Tree model.
- `estimatorParamMaps=paramGrid`: the hyperparameter grid defined earlier.
- `evaluator=RegressionEvaluator(...)`: evaluates performance using RMSE (Root Mean Squared Error).
- `trainRatio=0.8`: uses 80% of the training data for training and 20% for validation.

The model is trained using `tv.fit(train_data)`, and the total time taken is recorded by calculating the difference between the start and end timestamps. This approach helps identify the best model configuration while also tracking its computational cost.

```
start_time_tvs = time.time()
tv = TrainValidationSplit(
    estimator=pipeline,
    estimatorParamMaps=paramGrid,
    evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction",
    metricName="rmse"),
    trainRatio=0.8
)
model_tvs = tv.fit(train_data)
end_time_tvs = time.time()
```

```
start_time_tvs = time.time()

tvs = TrainValidationSplit(
    estimator=pipeline,
    estimatorParamMaps=paramGrid,
    evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="rmse"),
    trainRatio=0.8
)

model_tvs = tvs.fit(train_data)
end_time_tvs = time.time()
```

▶ (15) Spark Jobs

[v] Cross-Validation for Model Tuning and Performance Estimation

This code performs k-fold cross-validation to tune and train a Decision Tree Regressor in PySpark, while also measuring the training time. It begins by recording the start time using `time.time()`. A `CrossValidator` is then configured with:

- `estimator=pipeline`: the machine learning pipeline including feature assembly, scaling, and the decision tree.
- `estimatorParamMaps=paramGrid`: the hyperparameter grid to search over.
- `evaluator=RegressionEvaluator(...)`: evaluates model performance using RMSE.
- `numFolds=3`: performs 3-fold cross-validation, splitting the training data into 3 parts to ensure robust performance estimation.

The model is trained using `cv.fit(train_data)`, and the elapsed training time is calculated by the difference between start and end timestamps. This method helps identify the best parameter combination while reducing the risk of overfitting through repeated validation.

```
start_time_cv = time.time()
cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=paramGrid,
    evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction",
    metricName="rmse"),
    numFolds=3
)
model_cv = cv.fit(train_data)
```

```
end_time_cv = time.time()
```

```
02:27 PM (6m) 22

start_time_cv = time.time()

cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=paramGrid,
    evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="rmse"),
    numFolds=3
)

model_cv = cv.fit(train_data)
end_time_cv = time.time()
```

▶ (10) Spark Jobs

[vi] This code evaluates the performance of both the TrainValidationSplit (TVS) and CrossValidator (CV) models on the test data using RMSE and R^2 metrics. It also prints the execution time for each model training process.

```
pred_tvs = model_tvs.transform(test_data)
pred_cv = model_cv.transform(test_data)

evaluator_rmse = RegressionEvaluator(labelCol="label",
predictionCol="prediction", metricName="rmse")
evaluator_r2 = RegressionEvaluator(labelCol="label", predictionCol="prediction",
metricName="r2")

print("----- TVS Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_tvs))
print("R2:", evaluator_r2.evaluate(pred_tvs))
print("Execution Time (TVS): {:.2f} seconds".format(end_time_tvs - start_time_tvs))

print("\n----- CV Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_cv))
print("R2:", evaluator_r2.evaluate(pred_cv))
print("Execution Time (CV): {:.2f} seconds".format(end_time_cv - start_time_cv))
```

```
02:27 PM (1m) 23

pred_tvs = model_tvs.transform(test_data)
pred_cv = model_cv.transform(test_data)

evaluator_rmse = RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="rmse")
evaluator_r2 = RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="r2")

print("----- TVS Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_tvs))
print("R²:", evaluator_r2.evaluate(pred_tvs))
print("Execution Time (TVS): {:.2f} seconds".format(end_time_tvs - start_time_tvs))

print("\n----- CV Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_cv))
print("R²:", evaluator_r2.evaluate(pred_cv))
print("Execution Time (CV): {:.2f} seconds".format(end_time_cv - start_time_cv))

▶ (4) Spark Jobs
```

```
▶ (4) Spark Jobs

▶ pred_cv: pyspark.sql.dataframe.DataFrame
▶ pred_tvs: pyspark.sql.dataframe.DataFrame

----- TVS Model Evaluation -----
RMSE: 96777.44789842024
R²: 0.7329265125498214
Execution Time (TVS): 200.98 seconds

----- CV Model Evaluation -----
RMSE: 80703.61875728925
R²: 0.8142758134072039
Execution Time (CV): 364.80 seconds
```

[vii] Extracting Feature Importances from the Best Model

This code retrieves the best Decision Tree model from the TrainValidationSplit pipeline and extracts feature importances. It then maps each importance score to its corresponding feature name and prints the results, helping identify which input features most influenced the model's predictions.

```
best_model = model_tvs.bestModel.stages[-1] # Get the Decision Tree stage
feature_importances = best_model.featureImportances

# Print feature importance mapped to column names
print("\nFeature Importances:")
for i, col_name in enumerate(feature_cols):
    print(f"{col_name}: {feature_importances[i]:.4f}")
```

```
24 Python [ ] [ ] [ ]
best_model = model_tvsv.bestModel.stages[-1] # Get the Decision Tree stage
feature_importances = best_model.featureImportances

# Print feature importance mapped to column names
print("\nFeature Importances:")
for i, col_name in enumerate(feature_cols):
    print(f"{col_name}: {feature_importances[i]:.4f}")

Feature Importances:
HOMES_SOLD: 0.0169
NEW_LISTINGS: 0.0222
INVENTORY: 0.0159
MONTHS_OF_SUPPLY: 0.0063
MEDIAN_LIST_PRICE: 0.6916
MEDIAN_PPSF: 0.2395
PENDING_SALES: 0.0076
```

[Viii] Visualizing Predicted vs Actual Sale Prices

This step compares the predicted values from the Decision Tree model (TVS) with the actual labels by selecting both columns and converting them to a Pandas DataFrame for visualization. It then creates a scatter plot to show how closely the predictions align with the actual values. A red dashed diagonal line represents a perfect prediction (where predicted = actual), helping visually assess model accuracy — the closer the points are to this line, the better the model performance

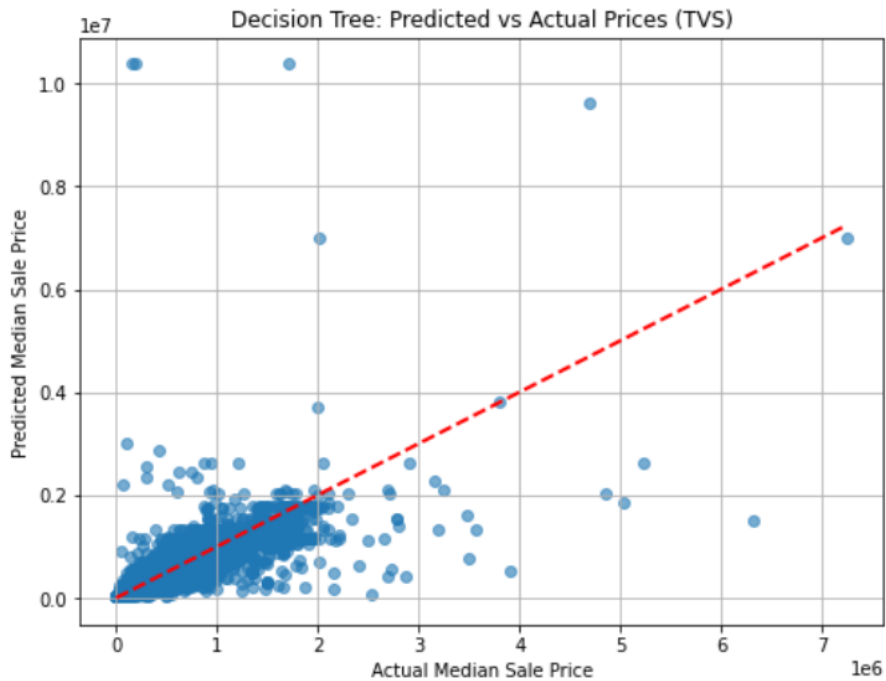
```
25 Python [ ] [ ] [ ]
# Select prediction and actual label from TVS model
prediction_vs_actual = pred_tvsv.select("prediction", "label")

# Show sample rows
prediction_vs_actual.show(10)
# Convert to pandas for visualization
pdf = prediction_vs_actual.toPandas()
import matplotlib.pyplot as plt

# Scatter plot of predictions vs actual values
plt.figure(figsize=(8, 6))
plt.scatter(pdf['label'], pdf['prediction'], alpha=0.6)
plt.plot([pdf['label'].min(), pdf['label'].max()],
         [pdf['label'].min(), pdf['label'].max()],
         color='red', linestyle='--', linewidth=2)
plt.xlabel("Actual Median Sale Price")
plt.ylabel("Predicted Median Sale Price")
plt.title("Decision Tree: Predicted vs Actual Prices (TVS)")
plt.grid(True)
plt.show()
```

```
▶ prediction_vs_actual: pyspark.sql.dataframe.DataFrame = [prediction: double, label: double]
```

```
+-----+-----+
| prediction | label |
+-----+-----+
| 26377.41095890411 | 40000.0 |
| 142068.38535031848 | 77900.0 |
| 201859.16003616637 | 295000.0 |
| 142068.38535031848 | 62000.0 |
| 201859.16003616637 | 235000.0 |
| 34878.0325732899 | 18500.0 |
| 100813.08193597561 | 84500.0 |
| 100813.08193597561 | 130000.0 |
| 231375.09023485784 | 105000.0 |
| 92824.78877005348 | 74300.0 |
+-----+-----+
only showing top 10 rows
```



Step 15: GBT Gradient-Boosted Trees

[i] Import Required Libraries

GBRegressor: Implements Gradient-Boosted Tree Regression, an ensemble learning technique that builds multiple trees sequentially to minimize error.

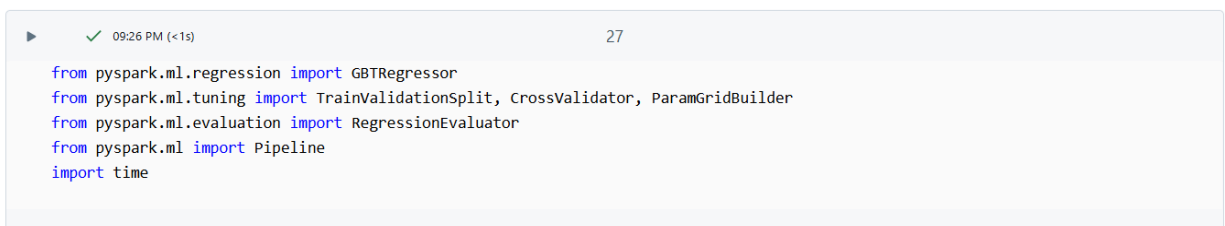
TrainValidationSplit & CrossValidator: Tools for hyperparameter tuning and model selection using validation or cross-validation.

ParamGridBuilder: Constructs a grid of hyperparameter values to search through.

RegressionEvaluator: Used to evaluate model performance (e.g., RMSE, R^2).

Pipeline: Chains multiple stages (e.g., feature processing + model) into a single workflow.

```
from pyspark.ml.regression import GBRegressor
from pyspark.ml.tuning import TrainValidationSplit, CrossValidator, ParamGridBuilder
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml import Pipeline
import time
```



```
from pyspark.ml.regression import GBRegressor
from pyspark.ml.tuning import TrainValidationSplit, CrossValidator, ParamGridBuilder
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml import Pipeline
import time
```

[ii] Define the GBT Regressor

labelCol: Column in your DataFrame that contains the target values.

featuresCol: Column containing the feature vectors (usually created using VectorAssembler).

```
gbt = GBRegressor(labelCol="label", featuresCol="normFeatures", maxIter=50)
pipeline_gbt = Pipeline(stages=[assembler, scaler, gbt])
```

```
▶ 09:26 PM (<1s) 28
gbt = GBRegressor(labelCol="label", featuresCol="normFeatures", maxIter=50)
pipeline_gbt = Pipeline(stages=[assembler, scaler, gbt])
```

[iii] Create a Parameter Grid

This defines combinations of hyperparameters to try.

maxDepth: Controls the maximum depth of each decision tree.

maxIter: Number of boosting iterations (trees).

```
paramGrid_gbt = ParamGridBuilder() \
    .addGrid(gbt.maxDepth, [3, 5]) \
    .addGrid(gbt.maxIter, [20, 50]) \
    .build()
```

```
▶ 09:26 PM (<1s) 29 Python
paramGrid_gbt = ParamGridBuilder() \
    .addGrid(gbt.maxDepth, [3, 5]) \
    .addGrid(gbt.maxIter, [20, 50]) \
    .build()
```

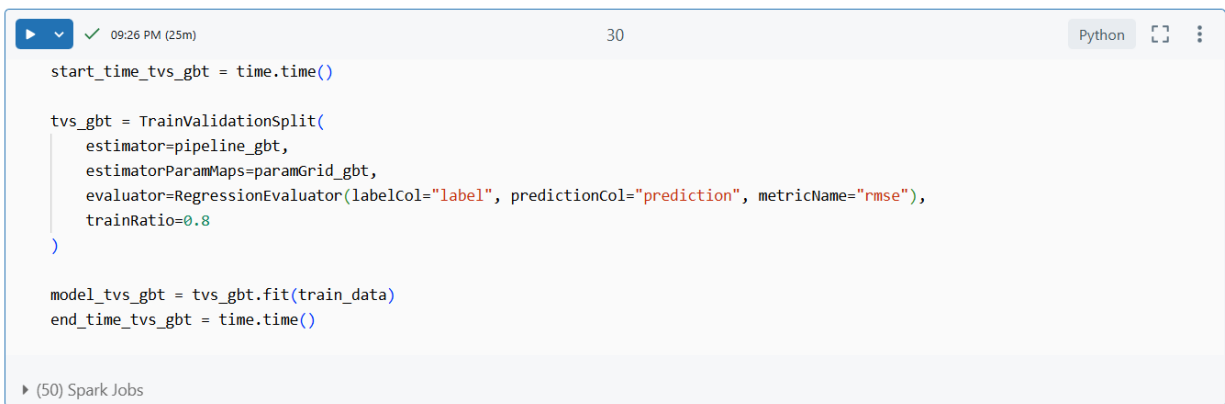
[IV] Training a GBT Regressor with TrainValidationSplit (TVS)

This code trains a Gradient-Boosted Tree (GBT) regression model using PySpark's TrainValidationSplit for hyperparameter tuning. It begins by recording the start time to measure training duration. A TrainValidationSplit object is created with a pipeline estimator (pipeline_gbt), a parameter grid (paramGrid_gbt), and a regression evaluator using RMSE as the metric. The training data is split 80/20, with 80% used for training and 20% for validation. The model is then trained using the .fit() method on train_data, and the end time is recorded to calculate how long the training took. This approach helps in selecting the best model configuration efficiently

```
start_time_tvs_gbt = time.time()

tvs_gbt = TrainValidationSplit(
    estimator=pipeline_gbt,
    estimatorParamMaps=paramGrid_gbt,
```

```
evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction",  
metricName="rmse"),  
trainRatio=0.8  
)  
  
model_tvsgbt = tvsgbt.fit(train_data)  
end_time_tvsgbt = time.time()
```



The screenshot shows a Jupyter Notebook interface with a code cell. The code defines a `TrainValidationSplit` object with a GBT pipeline as the estimator, a parameter grid for tuning, and an RMSE-based regression evaluator. It then fits the model on the training data and records the end time. The interface includes a status bar at the top showing a play button, a checkmark, the time '09:26 PM (25m)', the cell number '30', and the language 'Python'. At the bottom, it shows '(50) Spark Jobs'.

```
start_time_tvsgbt = time.time()  
  
tvsgbt = TrainValidationSplit(  
    estimator=pipeline_gbt,  
    estimatorParamMaps=paramGrid_gbt,  
    evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="rmse"),  
    trainRatio=0.8  
)  
  
model_tvsgbt = tvsgbt.fit(train_data)  
end_time_tvsgbt = time.time()
```

[V] Training a GBT Regressor with CrossValidation (CV)

This code performs training of a Gradient-Boosted Tree (GBT) regression model using PySpark's `CrossValidator` for more robust hyperparameter tuning. It starts by recording the current time to track the duration of the training process. A `CrossValidator` object is defined with the GBT pipeline (`pipeline_gbt`) as the estimator, a parameter grid (`paramGrid_gbt`) for tuning, and an RMSE-based regression evaluator to assess model performance. The cross-validation is configured with 3 folds, meaning the training data is split into three parts to validate the model on different subsets, ensuring a more reliable evaluation. The model is trained using the `.fit()` method on `train_data`, and the end time is captured to compute the total training time. This method offers better generalization by validating across multiple data splits.

```
start_time_cv_gbt = time.time()  
  
cv_gbt = CrossValidator(  
    estimator=pipeline_gbt,  
    estimatorParamMaps=paramGrid_gbt,
```



```

evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction",
metricName="rmse"),
numFolds=3
)

model_cv_gbt = cv_gbt.fit(train_data)
end_time_cv_gbt = time.time()

```



```

▶ 09:52 PM (47m) 31

start_time_cv_gbt = time.time()

cv_gbt = CrossValidator(
    estimator=pipeline_gbt,
    estimatorParamMaps=paramGrid_gbt,
    evaluator=RegressionEvaluator(labelCol="label", predictionCol="prediction", metricName="rmse"),
    numFolds=3
)

model_cv_gbt = cv_gbt.fit(train_data)
end_time_cv_gbt = time.time()

▶ (50) Spark Jobs

```

[VI] Comparative Analysis of GBT Model Performance

This section evaluates the performance of the trained GBT models from both TrainValidationSplit and CrossValidator. Predictions are generated on the test_data using the .transform() method for both models. The results are assessed using two evaluators: one for Root Mean Squared Error (RMSE) and another for R-squared (R^2), providing insight into the prediction accuracy and model fit. The evaluation results, along with the execution times for each training method, are printed. This comparison helps determine which tuning strategy—TVS or CV—offers better performance and computational efficiency.

```

pred_tvs_gbt = model_tvs_gbt.transform(test_data)
pred_cv_gbt = model_cv_gbt.transform(test_data)

print("----- GBT TVS Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_tvs_gbt))
print("R²:", evaluator_r2.evaluate(pred_tvs_gbt))
print("Execution Time (TVS): {:.2f} seconds".format(end_time_tvs_gbt -
start_time_tvs_gbt))

```

```
print("\n----- GBT CV Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_cv_gbt))
print("R²:", evaluator_r2.evaluate(pred_cv_gbt))
print("Execution Time (CV): {:.2f} seconds".format(end_time_cv_gbt - start_time_cv_gbt))
```

```
1 minute ago (1m) 32 Python
```

```
pred_tvs_gbt = model_tvs_gbt.transform(test_data)
pred_cv_gbt = model_cv_gbt.transform(test_data)

print("----- GBT TVS Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_tvs_gbt))
print("R²:", evaluator_r2.evaluate(pred_tvs_gbt))
print("Execution Time (TVS): {:.2f} seconds".format(end_time_tvs_gbt - start_time_tvs_gbt))

print("\n----- GBT CV Model Evaluation -----")
print("RMSE:", evaluator_rmse.evaluate(pred_cv_gbt))
print("R²:", evaluator_r2.evaluate(pred_cv_gbt))
print("Execution Time (CV): {:.2f} seconds".format(end_time_cv_gbt - start_time_cv_gbt))
```

► (4) Spark Jobs

```
► pred_cv_gbt: pyspark.sql.dataframe.DataFrame
► pred_tvs_gbt: pyspark.sql.dataframe.DataFrame

----- GBT TVS Model Evaluation -----
RMSE: 88461.05249824573
R²: 0.7768552612907269
Execution Time (TVS): 1509.93 seconds

----- GBT CV Model Evaluation -----
RMSE: 85666.11793459005
R²: 0.7907330616072379
Execution Time (CV): 2790.97 seconds
```

[VII] Feature Importance Analysis for GBT Model

This section extracts and displays feature importances from the best GBT model selected via TrainValidationSplit. The bestModel object refers to the final fitted pipeline, and the GBT regressor stage is accessed from the last stage of the pipeline (stages[-1]). The featureImportances attribute provides the relative importance of each feature, which is converted to an array for easier iteration. Each feature's name (from feature_cols) is printed alongside its corresponding importance score, helping to identify which input variables had the most influence on the model's predictions.

```
best_model_gbt = model_tvs_gbt.bestModel.stages[-1] # GBT stage
gbt_importances = best_model_gbt.featureImportances.toArray()

print("\nGBT Feature Importances:")
```

```
for i, col_name in enumerate(feature_cols):
    print(f'{col_name}: {gbt_importances[i]:.4f}')
```

```
▶ Just now (<1s) 33 Python [ ] ⋮

best_model_gbt = model_tvs_gbt.bestModel.stages[-1] # GBT stage
gbt_importances = best_model_gbt.featureImportances.toArray()

print("\nGBT Feature Importances:")
for i, col_name in enumerate(feature_cols):
    print(f'{col_name}: {gbt_importances[i]:.4f}')
```

GBT Feature Importances:
HOMES_SOLD: 0.0250
NEW_LISTINGS: 0.0487
INVENTORY: 0.0203
MONTHS_OF_SUPPLY: 0.0258
MEDIAN_LIST_PRICE: 0.5815
MEDIAN_PPSF: 0.2777
PENDING_SALES: 0.0211

[VIII] Visualization: Predicted vs Actual Values for GBT Model

This part of the code visualizes the performance of the GBT model trained with TrainValidationSplit by plotting predicted versus actual values for median sale prices. The predictions and actual labels are converted to a Pandas DataFrame for plotting with Matplotlib. A scatter plot is generated where each point represents a prediction-label pair, with a red dashed line indicating the ideal case where predicted values match actual values perfectly. The plot helps assess the accuracy and spread of predictions visually, making it easy to spot trends, outliers, or systematic errors.

```
pred_vs_actual_gbt = pred_tvs_gbt.select("prediction", "label").toPandas()

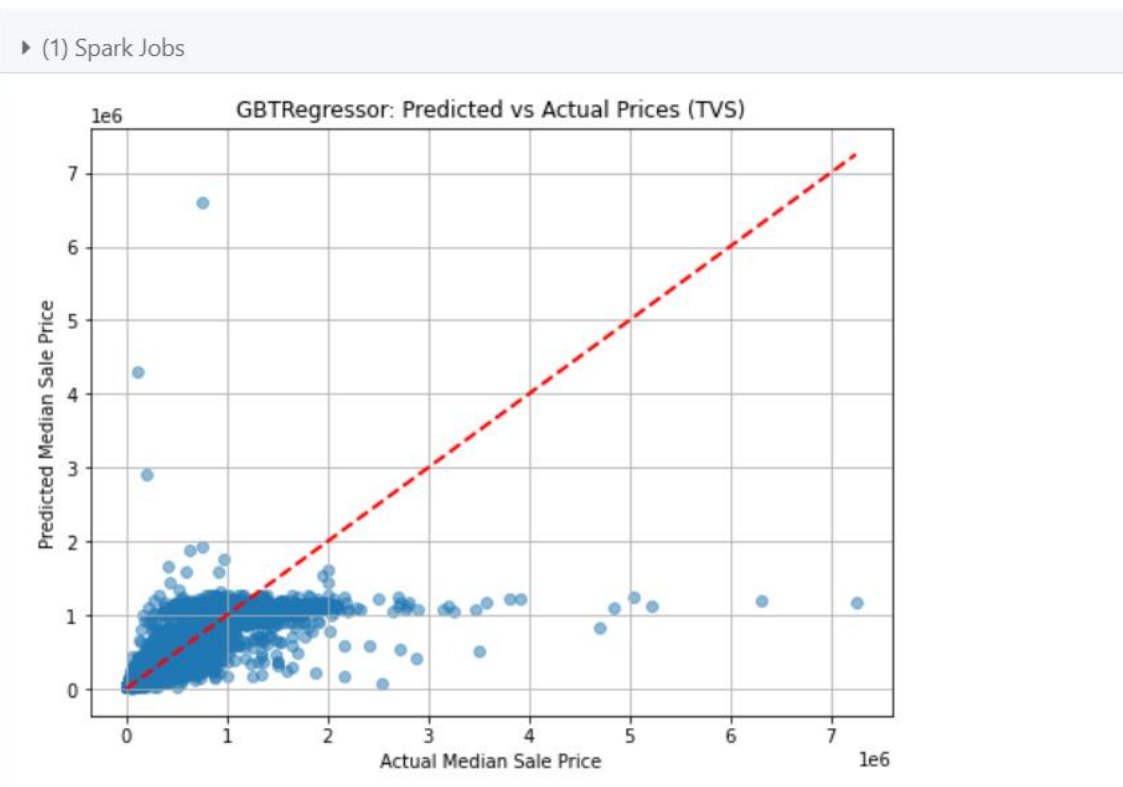
plt.figure(figsize=(8, 6))
plt.scatter(pred_vs_actual_gbt['label'], pred_vs_actual_gbt['prediction'], alpha=0.5)
plt.plot([pred_vs_actual_gbt['label'].min(), pred_vs_actual_gbt['label'].max()],
         [pred_vs_actual_gbt['label'].min(), pred_vs_actual_gbt['label'].max()],
         color='red', linestyle='--', linewidth=2)
plt.xlabel("Actual Median Sale Price")
plt.ylabel("Predicted Median Sale Price")
plt.title("GBTRegressor: Predicted vs Actual Prices (TVS)")
plt.grid(True)
plt.savefig("plot_output_gbt.png")
```



```
pred_vs_actual_gbt = pred_tvsgbt.select("prediction", "label").toPandas()

plt.figure(figsize=(8, 6))
plt.scatter(pred_vs_actual_gbt['label'], pred_vs_actual_gbt['prediction'], alpha=0.5)
plt.plot([pred_vs_actual_gbt['label'].min(), pred_vs_actual_gbt['label'].max()],
         [pred_vs_actual_gbt['label'].min(), pred_vs_actual_gbt['label'].max()],
         color='red', linestyle='--', linewidth=2)
plt.xlabel("Actual Median Sale Price")
plt.ylabel("Predicted Median Sale Price")
plt.title("GBRegressor: Predicted vs Actual Prices (TVS)")
plt.grid(True)
plt.savefig("plot_output_gbt.png")
```

▶ (1) Spark Jobs



Step 16 : Creating Binary Classification Labels Based on Median Sale Price

This step transforms the `MEDIAN_SALE_PRICE` column into a binary label column, where properties with a median sale price of \$500,000 or more are labeled as 1, and all others as 0. This prepares the dataset for a binary classification task, such as predicting whether a market is high-value or not.

```
from pyspark.sql.functions import when
```

```
# Create binary label column for classification
```

```
df_class = df.withColumn("label", when(col("MEDIAN_SALE_PRICE") >= 500000,  
1).otherwise(0))
```

```
▶ ✓ 1 hour ago (<1s) 35  
  
from pyspark.sql.functions import when  
  
# Create binary label column for classification  
df_class = df.withColumn("label", when(col("MEDIAN_SALE_PRICE") >= 500000, 1).otherwise(0))  
  
▶ df_class: pyspark.sql.dataframe.DataFrame = [PERIOD_BEGIN: date, PERIOD_END: date ... 58 more fields]
```

Selecting Features and Splitting Data for Classification

This step selects the relevant input features along with the label column to create the final classification dataset. Any rows with missing values are removed using `.dropna()`. The cleaned data is then randomly split into training (80%) and testing (20%) sets to prepare for model training and evaluation.

```
final_data_class = df_class.select(*feature_cols, "label").dropna()
```

```
train_data_class, test_data_class = final_data_class.randomSplit([0.8, 0.2], seed=42)
```

```
▶ ✓ 1 hour ago (<1s) 36  
  
final_data_class = df_class.select(*feature_cols, "label").dropna()  
train_data_class, test_data_class = final_data_class.randomSplit([0.8, 0.2], seed=42)  
  
▶ final_data_class: pyspark.sql.dataframe.DataFrame = [HOMES_SOLD: double, NEW_LISTINGS: double ... 6 more fields]  
▶ test_data_class: pyspark.sql.dataframe.DataFrame = [HOMES_SOLD: double, NEW_LISTINGS: double ... 6 more fields]  
▶ train_data_class: pyspark.sql.dataframe.DataFrame = [HOMES_SOLD: double, NEW_LISTINGS: double ... 6 more fields]
```

Building a Gradient-Boosted Tree Classification Pipeline with Hyperparameter Tuning

This section defines a machine learning pipeline for binary classification using `GBTCClassifier`. The pipeline includes three main stages:

1. Feature Assembling – Combines selected features into a single feature vector.

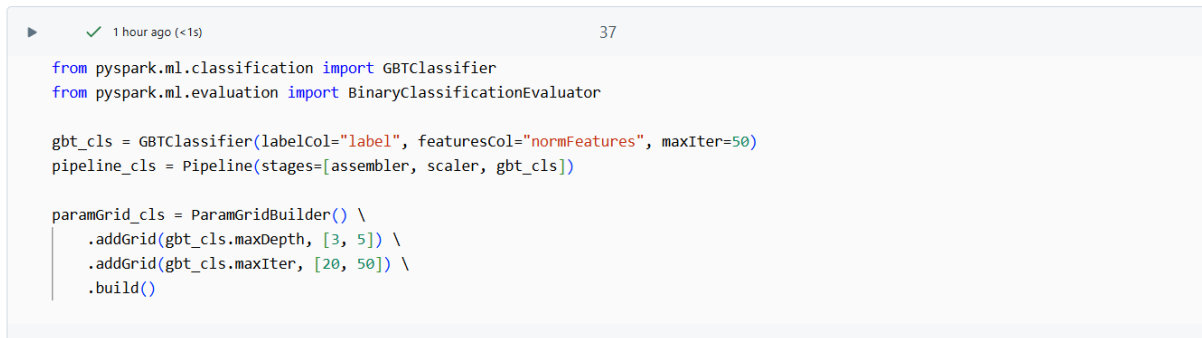
2. Feature Scaling – Normalizes features for optimal model performance.
3. Model Training – Trains a Gradient-Boosted Tree Classifier on the processed features.

A parameter grid is also defined to tune maxDepth and maxIter using cross-validation or train-validation split.

```
from pyspark.ml.classification import GBTClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator

gbt_cls = GBTClassifier(labelCol="label", featuresCol="normFeatures", maxIter=50)
pipeline_cls = Pipeline(stages=[assembler, scaler, gbt_cls])

paramGrid_cls = ParamGridBuilder() \
    .addGrid(gbt_cls.maxDepth, [3, 5]) \
    .addGrid(gbt_cls.maxIter, [20, 50]) \
    .build()
```



```
from pyspark.ml.classification import GBTClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator

gbt_cls = GBTClassifier(labelCol="label", featuresCol="normFeatures", maxIter=50)
pipeline_cls = Pipeline(stages=[assembler, scaler, gbt_cls])

paramGrid_cls = ParamGridBuilder() \
    .addGrid(gbt_cls.maxDepth, [3, 5]) \
    .addGrid(gbt_cls.maxIter, [20, 50]) \
    .build()
```

Training the Classification Model with Train-Validation Split

This section applies TrainValidationSplit to train the Gradient-Boosted Tree classifier using 80% of the training data for model fitting and 20% for validation during hyperparameter tuning. The evaluator uses AUC (areaUnderROC) to measure model performance. Training time is also recorded to monitor the model fitting duration.

```
tv_cls = TrainValidationSplit(
    estimator=pipeline_cls,
    estimatorParamMaps=paramGrid_cls,
```

```
evaluator=BinaryClassificationEvaluator(labelCol="label",
metricName="areaUnderROC"),
trainRatio=0.8
)
```

```
start_tvs_cls = time.time()
model_tvs_cls = tvs_cls.fit(train_data_class)
end_tvs_cls = time.time()
```



The screenshot shows a Jupyter Notebook interface. At the top, there is a status bar indicating '1 hour ago (25m)' and a cell number '38'. The code in the cell is as follows:

```
tvs_cls = TrainValidationSplit(
    estimator=pipeline_cls,
    estimatorParamMaps=paramGrid_cls,
    evaluator=BinaryClassificationEvaluator(labelCol="label", metricName="areaUnderROC"),
    trainRatio=0.8
)

start_tvs_cls = time.time()
model_tvs_cls = tvs_cls.fit(train_data_class)
end_tvs_cls = time.time()
```

At the bottom of the cell, there is a status bar indicating '(50) Spark Jobs'.

Training the Classification Model with Cross-Validation

This step uses a CrossValidator to train and evaluate the Gradient-Boosted Tree classification pipeline with 3-fold cross-validation. The model is assessed using Area Under the ROC Curve (AUC) as the evaluation metric. This process helps select the best combination of hyperparameters from the parameter grid. The training duration is also measured for performance tracking.

```
cv_cls = CrossValidator(
    estimator=pipeline_cls,
    estimatorParamMaps=paramGrid_cls,
    evaluator=BinaryClassificationEvaluator(labelCol="label",
metricName="areaUnderROC"),
    numFolds=3
)
```

```
start_cv_cls = time.time()
model_cv_cls = cv_cls.fit(train_data_class)
end_cv_cls = time.time()
```

```
55 minutes ago (54m) 39

cv_cls = CrossValidator(
    estimator=pipeline_cls,
    estimatorParamMaps=paramGrid_cls,
    evaluator=BinaryClassificationEvaluator(labelCol="label", metricName="areaUnderROC"),
    numFolds=3
)

start_cv_cls = time.time()
model_cv_cls = cv_cls.fit(train_data_class)
end_cv_cls = time.time()

▶ (50) Spark Jobs
```

Evaluating Classification Models and Comparing TVS vs Cross-Validation Performance

This section evaluates both the Train-Validation Split (TVS) and Cross-Validation (CV) trained models on the test dataset using AUC (Area Under the ROC Curve). It outputs both the AUC scores and the training execution times to compare model performance and training efficiency across the two tuning strategies.

```
pred_tvs_cls = model_tvs_cls.transform(test_data_class)
pred_cv_cls = model_cv_cls.transform(test_data_class)

evaluator_auc = BinaryClassificationEvaluator(labelCol="label",
metricName="areaUnderROC")

print("----- GBTClassifier TVS Evaluation -----")
print("AUC:", evaluator_auc.evaluate(pred_tvs_cls))
print("Execution Time (TVS): {:.2f} sec".format(end_tvs_cls - start_tvs_cls))

print("\n----- GBTClassifier CV Evaluation -----")
print("AUC:", evaluator_auc.evaluate(pred_cv_cls))
print("Execution Time (CV): {:.2f} sec".format(end_cv_cls - start_cv_cls))
```



```
1 minute ago (52s) 40 Python
pred_tvs_cls = model_tvs_cls.transform(test_data_class)
pred_cv_cls = model_cv_cls.transform(test_data_class)

evaluator_auc = BinaryClassificationEvaluator(labelCol="label", metricName="areaUnderROC")

print("----- GBTClassifier TVS Evaluation -----")
print("AUC:", evaluator_auc.evaluate(pred_tvs_cls))
print("Execution Time (TVS): {:.2f} sec".format(end_tvs_cls - start_tvs_cls))

print("\n----- GBTClassifier CV Evaluation -----")
print("AUC:", evaluator_auc.evaluate(pred_cv_cls))
print("Execution Time (CV): {:.2f} sec".format(end_cv_cls - start_cv_cls))

▶ (8) Spark Jobs
```

```
▶ pred_cv_cls: pyspark.sql.dataframe.DataFrame
▶ pred_tvs_cls: pyspark.sql.dataframe.DataFrame
----- GBTClassifier TVS Evaluation -----
AUC: 0.9939654493703334
Execution Time (TVS): 1493.98 sec

----- GBTClassifier CV Evaluation -----
AUC: 0.9939570120832761
Execution Time (CV): 3210.61 sec
```

Extracting Feature Importances from the Best GBT Classification Model

This section retrieves the best model from the Train-Validation Split (TVS) pipeline and extracts the feature importance scores from the trained GBTClassifier. These scores help interpret which input features had the greatest influence on the model's predictions, offering valuable insights into the data.

```
best_model_cls = model_tvs_cls.bestModel.stages[-1]
importance_cls = best_model_cls.featureImportances.toArray()

print("\nGBTClassifier Feature Importances:")
for i, col_name in enumerate(feature_cols):
    print(f"{col_name}: {importance_cls[i]:.4f}")
```

```
▶ Just now (<1s) 41 Python

best_model_cls = model_tvs_cls.bestModel.stages[-1]
importance_cls = best_model_cls.featureImportances.toArray()

print("\nGBClassifier Feature Importances:")
for i, col_name in enumerate(feature_cols):
    print(f"{col_name}: {importance_cls[i]:.4f}")

GBClassifier Feature Importances:
HOMES_SOLD: 0.0264
NEW_LISTINGS: 0.0271
INVENTORY: 0.0135
MONTHS_OF_SUPPLY: 0.0194
MEDIAN_LIST_PRICE: 0.7734
MEDIAN_PPSF: 0.1247
PENDING_SALES: 0.0154
```

Step 17: Run the Same code in Spark-submit

Open gitbash and copy the python code from local machine to the cluster by doing SCP

```
Sirisha@LAPTOP-8FCHOK16 MINGW64 ~ (main)
$ scp ml_regression_models.py smahesh4@144.24.13.0:~
smaresh4@144.24.13.0's password:
ml_regression_models.py 100% 14KB 533.3KB/s 00:00
```

After SCP check in another terminal by doing ls after login to ssh

Step 18: Run the spark-submit code

```
spark-submit ml_regression_model.py
```

Spark-submit Screen-shots

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|PERIOD_BEGIN|PERIOD_END|PERIOD_DURATION|REGION_TYPE|REGION_TYPE_ID|TABLE_ID|IS_SEASONALLY_ADJUSTED|REGION|CITY|STA
TE|STATE_CODE|PROPERTY_TYPE|PROPERTY_TYPE_ID|MEDIAN_SALE_PRICE|MEDIAN_SALE_PRICE_MOM|MEDIAN_SALE_PRICE_YOY|MEDIAN_LIST_PRICE|M
EDIAN_LIST_PRICE_MOM|MEDIAN_LIST_PRICE_YOY|MEDIAN_PPSF|MEDIAN_PPSF_MOM|MEDIAN_PPSF_YOY|MEDIAN_LIST_PPSF|MEDIAN_LIST_PPSF_MOM|MEDIAN
_LIST_PPSF_YOY|HOMES_SOLD|HOMES_SOLD_MOM|HOMES_SOLD_YOY|PENDING_SALES|PENDING_SALES_MOM|PENDING_SALES_YOY|NEW_LISTINGS|NEW_LISTINGS_M
OM|NEW_LISTINGS_YOY|INVENTORY|INVENTORY_MOM|INVENTORY_YOY|MONTHS_OF_SUPPLY|MONTHS_OF_SUPPLY_MOM|MONTHS_OF_SUPPLY_YOY|MEDIAN_DOM|MED
IAN_DOM_MOM|MEDIAN_DOM_YOY|AVG_SALE_TO_LIST|AVG_SALE_TO_LIST_MOM|AVG_SALE_TO_LIST_YOY|SOLD_ABOVE_LIST|SOLD_ABOVE_LIST_MOM|SOLD_ABOVE
_LIST_YOY|PRICE_DROPS|PRICE_DROPS_MOM|PRICE_DROPS_YOY|OFF_MARKET_IN_TWO_WEEKS|OFF_MARKET_IN_TWO_WEEKS_MOM|OFF_MARKET_IN_TWO_WEEKS_YOY
|PARENT_METRO_REGION|PARENT_METRO_REGION_METRO_CODE|LAST_UPDATED|SOURCE_FILE|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2023-12-01|2023-12-31|30|county|5|3220|false|Sawyer County, WI|null|Wiscons
in|WI|Single Family Res...|6|260000|-0.1544715447|-0.1018998273|397450|
0.98725|0.7287951283|209.573791349|0.07354130799|0.1997483161|213.969853676|0.1807700955|
0.09552565082|19|-0.2083333333|-0.3448275862|13|-0.2777777778|0|8|-0.6190476
19|0.1428571429|75|-0.09638554217|-0.08536585366|3.9|0.4|1.1|81|
37|23|0.9736825207|0.0194297376|0.009115501788|0.3157894737|0.2324561404|0.1
08892922|0.16|0.02746987952|0.07463414634|0.3076923077|-0.02564102564|0.07692307692
|Wisconsin nonmetr...|NA|2025-04-18 14:40:...|county_market_tra...|
| 2015-12-01|2015-12-31|30|county|5|1583|false|Stone County, MS|null|Mississip
pi|MS|Single Family Res...|6|120000|0.0480349345|0.3961605585|119900|
0.2002002002|0.2002002002|81.5|0.2127023066|0.4195888102|76.388888889|0.4413719275|
0.01583820663|11|0.8333333333|0.8333333333|10|1|0.6666666667|7|-0.22222222
22|0.1666666667|71|-0.07792207792|-0.1647058824|6.5|-6.3|-7.7|167|
```

```
only showing top 5 rows
```

DESCRIPTION	words
Sawyer County, WI Wisconsin Single Family Residential	[sawyer, county, wisconsin, single, family, residential]
Stone County, MS Mississippi Single Family Residential	[stone, county, mississippi, single, family, residential]
Marion County, MS Mississippi Single Family Residential	[marion, county, mississippi, single, family, residential]
Blount County, AL Alabama All Residential	[blount, county, alabama, all, residential]
Lehigh County, PA Pennsylvania All Residential	[lehigh, county, pennsylvania, all, residential]

```
only showing top 5 rows
```

words	filtered
-------	----------

topic	termIndices	termWeights
0	[1, 0, 6, 2, 9, 8...]	[0.20529851464311...]
1	[0, 1, 3, 2, 4, 5...]	[0.18356897594964...]
2	[1, 0, 13, 14, 15...]	[0.17601032335254...]
3	[0, 4, 5, 1, 26, ...]	[0.19619570844772...]
4	[5, 4, 2, 8, 7, 0...]	[0.21707351527609...]

```

Topic 0: ['county', 'residential', 'condo', 'family', 'townhouse', 'unit', 'multi', 'virginia', 'new', 'ohio']
Topic 1: ['residential', 'county', 'single', 'family', 'area', 'metro', 'texas', 'tennessee', 'iowa', 'south']
Topic 2: ['county', 'residential', 'carolina', 'north', 'georgia', 'family', 'single', 'michigan', 'louisiana', 'condo']
Topic 3: ['residential', 'area', 'metro', 'county', 'pennsylvania', 'city', 'condo', 'family', 'north', 'oklahoma']
Topic 4: ['metro', 'area', 'family', 'unit', 'multi', 'residential', 'condo', 'townhouse', 'san', 'california']
TVS Model Evaluation

```

Decision Tree output

```

----- TVS Model Evaluation -----
RMSE: 113042.9498554298
R²: 0.6830126936958228
Execution Time (TVS): 388.03 seconds

----- CV Model Evaluation -----
RMSE: 112700.58156701337
R²: 0.6849298775103057
Execution Time (CV): 646.78 seconds

```

```

Feature Importances:
HOMES_SOLD: 0.0294
NEW_LISTINGS: 0.0426
INVENTORY: 0.0482
MONTHS_OF_SUPPLY: 0.0247
MEDIAN_LIST_PRICE: 0.2158
MEDIAN_PPSF: 0.6163
PENDING_SALES: 0.0232

```

GBT output

```
----- GBT TVS Model Evaluation -----  
RMSE: 116349.98178720113  
R²: 0.664194699614528  
Execution Time (TVS): 567.51 seconds  
  
----- GBT CV Model Evaluation -----  
RMSE: 116349.70389536627  
R²: 0.664196303696251  
Execution Time (CV): 1066.09 seconds  
  
GBT Feature Importances:  
HOMES_SOLD: 0.0396  
NEW_LISTINGS: 0.0405  
INVENTORY: 0.0484  
MONTHS_OF_SUPPLY: 0.0699  
MEDIAN_LIST_PRICE: 0.1965  
MEDIAN_PPSF: 0.5806  
PENDING_SALES: 0.0245
```

References:

- [1] Using Machine Learning and Big Data to Predict Real Estate Returns." *Due Diligence*, UF Warrington College of Business, 19 Mar. 2025, warrington.ufl.edu/due-diligence/2025/03/19/machine-learning-big-data-predict-real-estate-returns/.
- [2] Singh, Yashraj, Vivek Kumar, and Asim Ahmad. "Real Estate Price Prediction: A Comprehensive Review." *International Journal for Research in Applied Science & Engineering Technology*, vol. 12, no. 2, Feb. 2024, pp. 752-57, doi:10.22214/ijraset.2024.58428.
- [3] Git Hub link : <https://github.com/sirishavivek2/CIS5560>
- [4] Redfin Data Centre. Redfin, www.redfin.com/news/data-center/
- [5] Data Source URL <https://www.kaggle.com/datasets/thuynyle/redfin-housing-market-data>

=====END=====

